

Working with current models



DANIEL VALDENEGRO

*Leverhulme Centre for Demographic Science & Nuffield
College
University of Oxford*

Large Language Models for Health and Social Science
Research, June-July 2026



Today's question: not just *what LLMs are*, but *how to work with them responsibly as a researcher*.

1. Foundation models: the conceptual frame (Bommasani et al.)
2. The model landscape today
3. What current models actually look like (interactive)
4. Model choice as a research decision
5. Hosted APIs vs. locally hosted models
6. Data governance
7. Documenting model use

Throughout: the same reliability/validity distinction from Lecture 2 applies to *tooling* decisions, not just model outputs.

What is a “foundation model” ?

Bommasani et al. (2021) coin the term to describe a shift already underway in NLP and beyond.

Definition

A **foundation model** is any model trained on broad data (typically via self-supervision) at scale, that can be *adapted* to a wide range of downstream tasks.

This is a generalisation of what Lecture 2 called self-supervised next-token prediction: the same underlying object now sits underneath classification, extraction, summarisation, dialogue, code generation, and more.

Why a new term at all? The authors argue that scale changes the *kind* of thing this is, not just its size — a single model becomes genuinely general-purpose infrastructure.

Emergence: capabilities that weren't explicitly trained

One of the paper's central claims: as foundation models scale, they exhibit **emergent behaviour** — capabilities that were not explicitly targeted by the training objective and are difficult to predict in advance.

- ▶ Recall from Lecture 2: the training objective is just next-token log-likelihood.
- ▶ Nothing in that objective says “learn arithmetic” or “learn to translate” or “follow multi-step instructions.”
- ▶ Yet these behaviours appear, often quite abruptly, as scale increases.

Consequence for practice: you cannot fully characterise what a foundation model can do by reading its training objective or spec sheet. Capabilities are discovered empirically, sometimes *after* deployment — which is itself a risk, not just a curiosity.

The second central claim, and arguably the more consequential one for research practice.

Homogenization

Because the same small set of foundation models gets adapted into many downstream systems, any flaw, bias, or blind spot in the foundation model **propagates** into every system built on top of it.

- ▶ A bias in a widely-used foundation model is no longer one dataset's problem — it is inherited by every classifier, annotator, or chatbot fine-tuned or prompted from it.
- ▶ This is a direct structural analogue of Lecture 2's *inherited bias* failure mode (Slide 40), but at the scale of an entire research field rather than one paper.
- ▶ Benchmarks are affected too: if everyone evaluates against the same handful of models, blind spots in evaluation become shared blind spots in the literature.

The paper's title is not decoration — it insists the two are inseparable.

Opportunities

- ▶ Shared infrastructure lowers the cost of building new applications
- ▶ Capabilities transfer across tasks and domains
- ▶ Rapid prototyping for under-resourced research questions

Risks

- ▶ Homogenized failure modes across many systems
- ▶ Emergent behaviour is hard to audit or predict
- ▶ Concentration of power in a small number of model providers

Bridge forward. Every choice you make today — which model, which API, whether to fine-tune, how to document it — is a choice about how much of this opportunity/risk bundle you are importing into your own research.

The model landscape: orientation, not a fixed map

Any snapshot of “which models exist” is stale within months. What’s stable is the set of **axes** along which models differ.

- ▶ **Access:** closed, API-only (e.g. GPT, Claude, Gemini) vs. open-weight (e.g. Llama, Mistral, Qwen, DeepSeek)
- ▶ **Size / cost tier:** frontier flagship vs. smaller, cheaper, faster variants of the same family
- ▶ **Behaviour mode:** standard chat/completion vs. “reasoning” models that generate extended intermediate computation before answering
- ▶ **Modality:** text-only vs. multimodal (text, image, audio — more on this shortly)
- ▶ **Context window:** how many tokens of prefix the model can condition on (Lecture 2, Slide 13)

Practical stance: don’t memorise today’s leaderboard. Learn to place any new model on these axes.

Bommasani et al.'s homogenization argument explains a structural feature of the landscape itself.

- ▶ A small number of organisations can afford frontier-scale pretraining runs.
- ▶ Most researchers — and most downstream products — adapt *one of a handful of* foundation models, rather than training their own.
- ▶ This is precisely why model *choice* matters as much as model *use*: you are opting into one of a small number of shared dependencies.

Open-weight models partially mitigate this: they let you inspect, modify, and host the model yourself — at the cost of typically lagging frontier closed models on many tasks, and requiring your own infrastructure (Section 5).

Training Compute-Optimal Large Language Models

The paper asks a deceptively simple question: for a fixed compute budget, how should you split it between **model size** and **training data**?

Prior practice. Many large models were trained with parameter counts that were too large relative to the amount of training data, given the compute spent.

Main result. Model size and training tokens should scale roughly *equally* as compute increases. The authors train Chinchilla (70B parameters) on $4\times$ more data than a comparable prior model, using the same compute, and it outperforms models several times larger.

Why Chinchilla matters for model choice

This is not a historical footnote — it is a direct challenge to the intuition “bigger model = better.”

- ▶ Two models with the same parameter count can differ enormously in quality depending on how much (and what) data they were trained on.
- ▶ Two models with very different parameter counts can perform comparably if the smaller one was trained compute-optimally.
- ▶ **Consequence:** parameter count, on its own, tells you almost nothing about a model's likely performance on your task.

Bridge back to Lecture 2: recall the training objective

$\hat{\theta} = \arg \max_{\theta} \sum_t \log p_{\theta}(x_t \mid x_{<t})$. Chinchilla is about *how the compute budget behind that $\arg \max$ gets allocated* — not a change to the objective itself.

So how should you actually read a model's spec sheet?

- ▶ **Ignore** parameter count as a standalone quality signal.
- ▶ **Look for** task-relevant benchmark performance — but recall the contamination caveat (Lecture 2, Slide 27): benchmark numbers may partly reflect memorisation, not generalisation.
- ▶ **Look for** context window size, modality support, and release/version date — these are more diagnostic for research use than raw scale.
- ▶ **Test on your own data.** A small sample of your actual task, run against 2–3 candidate models, is worth more than any leaderboard.

Reflection question. If homogenization means most of the field converges on the same few models, what does that imply for how much independent verification any single benchmark claim deserves?

Before I explain anything — quick poll.

When you upload an image to ChatGPT and ask “what’s in this picture,” and then separately ask it to “generate an image of a cat in a spacesuit”:

- ▶ Is the same neural network doing both things?
- ▶ Is there *one* set of weights, or several?
- ▶ When it searches the web for you, is that the language model itself, or something else?

Hold your answers. We’ll come back to this slide at the end of the section.

The short answer: it's a system, not a model

What feels like “one assistant” is, almost always, an **orchestrated system** of several distinct components, often with entirely different architectures and training objectives:

1. A **text (language) model** — the next-token predictor from Lecture 2, handling reasoning, drafting, and conversation.
2. An **image understanding component** — lets the text model “see” an uploaded image.
3. An **image generation component** — a separate model entirely, usually diffusion-based.
4. An **orchestration layer** — decides which of the above to call, in what order, and how to combine their outputs.

This is a direct, concrete instance of Bommasani et al.'s point: a foundation model is rarely deployed alone — it sits inside a larger adapted system.

How image *ingestion* works: connecting pixels to words

How does a next-token predictor, trained on text, “read” an image?

The key idea: a shared embedding space. Models like CLIP (Contrastive Language–Image Pretraining) are trained on paired data: (image, caption).

- ▶ An **image encoder** maps an image to a vector $z_{\text{img}} \in \mathbb{R}^d$.
- ▶ A **text encoder** maps its caption to a vector $z_{\text{txt}} \in \mathbb{R}^d$.
- ▶ Training pulls z_{img} and z_{txt} **close together** in the same vector space when they belong to a true pair, and pushes them apart otherwise (a *contrastive* objective).

Result: images and text end up living in a *common geometric space*. An uploaded image can be converted into something that looks, to the language model, like more tokens it can attend to — even though it never saw a pixel during its own pretraining.

Why this matters: images become “soft tokens”

- ▶ The image encoder's output vectors are projected into the same embedding space the language model uses for text tokens.
- ▶ Practically: the image is chopped into patches, each patch is encoded, and the resulting vectors are fed into the language model's context window *alongside* the text tokens of your prompt.
- ▶ The causal/attention mechanism from Lecture 2 (Slide 9) then treats these image-derived vectors just like any other tokens it attends to.

Consequence for research use: an LLM's ability to describe or reason about an image is only as good as (a) the image encoder's training data, and (b) how well image and text embeddings were aligned. Errors here look like confident misdescription — a visual analogue of hallucination (Lecture 2, Slide 38).

Image generation is a completely different architecture and training objective from next-token prediction — there is no “next pixel” to predict in the same sequential sense.

Core idea: learn to reverse noise.

1. **Forward process (training):** take a real image, add a small amount of random noise, then more, then more, until it's pure static, over T steps.
2. **The model's job:** at each noise level t , predict the noise that was added, so it can be subtracted.
3. **Generation (inference):** start from pure random noise, and repeatedly apply the trained model to remove a little noise at a time, over T steps, until a coherent image emerges.

Schematically: x_T (noise) $\rightarrow x_{T-1} \rightarrow \dots \rightarrow x_1 \rightarrow x_0$ (image).

Text-to-image: conditioning the denoiser

A plain diffusion model just produces *some* plausible image. To make it follow a text prompt, the denoising step is **conditioned** on a text embedding — often from the very same kind of text encoder used in CLIP.

$$p_{\theta}(x_{t-1} \mid x_t, c), \quad c = \text{embedding of your prompt}$$

- ▶ At every denoising step, the model asks: “which way should I nudge this noisy image to look more like *both* (a) a real image, and (b) something matching this text embedding c ?”
- ▶ This is why the same word (“cat,” “spacesuit”) reliably steers generation — text and image share a common representational space, exactly as in ingestion.

Note the asymmetry: the language model in your chat is not drawing the picture. It writes a text prompt, which is handed to an entirely separate diffusion model.

The experience of a single coherent assistant is produced by an **orchestration layer** sitting above all the individual models.

- ▶ The language model itself is typically trained to recognise when a request needs a **tool** — image generation, web search, code execution, retrieval — and to emit a structured “call” rather than trying to do it directly.
- ▶ A surrounding system routes that call to the right component (diffusion model, search API, code sandbox), gets a result back, and feeds it back into the language model’s context window as more tokens.
- ▶ The language model then continues generating, now conditioned on that tool’s output — exactly the context-window mechanism from Lecture 2 (Slide 13), just with a wider variety of things allowed *into* the context.

Back to the opening questions:

- ▶ **Same network for image understanding and image generation?** No — typically a separate encoder (CLIP-like) for understanding, and a separate diffusion model for generation.
- ▶ **One set of weights?** No — at minimum: a language model, an image encoder, a diffusion model, and often a separate retrieval/search system, all with independently trained parameters.
- ▶ **Is web search “the model”?** No — it’s an external tool the language model has learned to call, not a capability of its own weights.

The takeaway, in Bommasani’s terms: what you experience as “an AI model” is a foundation model *plus* an adaptation-and-orchestration layer around it. When you cite “I used ChatGPT,” you are underspecifying a whole stack — which is exactly the documentation problem we return to at the end of the lecture.

Model choice is a methodological decision, not a preference

In classical statistics, choosing a model family (linear vs. logistic, fixed vs. random effects) is a methods-section decision, justified and reported.

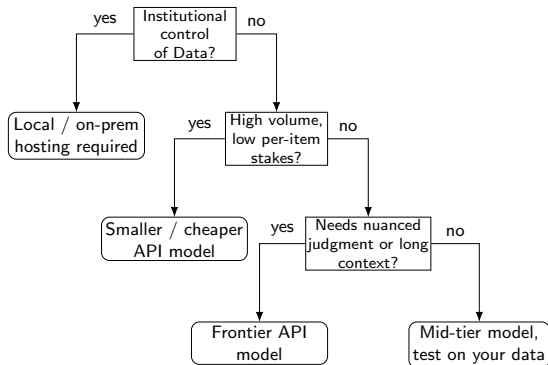
Choosing which LLM to use should be treated the same way. It affects:

- ▶ the validity of your results (different models, different error patterns);
- ▶ the reproducibility of your study (Lecture 2, Slide 44 — models are moving targets);
- ▶ the cost and feasibility of your research design;
- ▶ what data governance obligations you take on (Section 6, shortly).

Framing for today: treat “which model” as an explicit, justifiable methodological choice — not an afterthought resolved by whatever tool is open in your browser.

Before picking a model, answer these for your specific task:

1. **Task type:** classification/annotation (Lecture 2's Ziems, Gilardi case studies) vs. open-ended generation vs. structured extraction — these stress different capabilities.
2. **Data sensitivity:** can this data leave your institution's control at all? (Section 6)
3. **Volume and cost:** thousands of API calls at a few cents each adds up fast; local hosting has a different cost shape (hardware, not per-call).
4. **Context length needed:** one tweet vs. a 40-page policy document is a different requirement entirely.
5. **Reproducibility requirements:** will you need the *exact* same behaviour in six months, for review or replication?



This is a starting heuristic, not a substitute for testing on your own data.

It's tempting to resolve model choice by checking a leaderboard. Treat this with the same caution as any single evaluation metric (Lecture 2, Slide 34).

- ▶ **Contamination:** if a benchmark's text overlapped with a model's pretraining data, high scores partly reflect memorisation, not generalisation (Lecture 2, Slide 27).
- ▶ **Homogenization risk (Bommasani):** if most of the field evaluates on the same handful of benchmarks against the same handful of models, a blind spot in the benchmark becomes an invisible blind spot across the whole literature.
- ▶ **Task mismatch:** a model that tops a general reasoning leaderboard may be mediocre at, say, extracting structured fields from clinical notes — these are different skills.

Practical rule: benchmarks narrow the shortlist. Your own held-out sample, evaluated with a metric that matches your actual question (Lecture 2, Slide 34), makes the final decision.

A lightweight way to make “model choice” a reportable, defensible step in your workflow:

1. Draw a small stratified sample of your actual task data (50–200 items, depending on task variability).
2. Run 2–3 candidate models with *identical* prompts and parameters.
3. Score against the metric appropriate to your task (accuracy/F1/ κ for classification; human or LLM-as-judge rating for open-ended output — Lecture 2, Slide 33–34).
4. Record cost and latency alongside accuracy — the “best” model on accuracy alone may be impractical at your required volume.
5. Report this comparison, briefly, in your methods — this *is* the documentation habit we return to in Section 7.

Two ways to run a foundation model

Once you've chosen a model (Section 4), you still need to decide *how you access it*.
Two broad options:

- ▶ **Hosted API:** send requests over the internet to a provider's servers; you never see or hold the model weights.
- ▶ **Locally hosted:** download the model weights and run inference yourself, on your own (or your institution's) hardware.

This is not purely a technical choice — it interacts directly with data governance (Section 6), cost structure, and reproducibility (Lecture 2, Slide 44).

Important constraint: locally hosted, in practice, means **open-weight models**.

Frontier closed models (GPT, Claude, Gemini) are API-only by design — you cannot download their weights.

Hosted APIs: how it works mechanically

At its simplest, an API call is:

your prompt (text/image) $\longrightarrow$ provider's server $\longrightarrow$ model output

Parameters you can typically control:

- ▶ **System prompt / instructions:** sets persistent context or task framing (Lecture 2, Slide 15's prompt engineering)
- ▶ **Temperature:** controls randomness of sampling — lower means more consistent, deterministic-*ish* output (recall Gilardi et al., $T = 0.2$, Lecture 2 Slide 21)
- ▶ **Max tokens:** caps output length (and cost)
- ▶ **Structured output / JSON mode:** constrains the model to emit schema-conforming output (Lecture 2, Slide 37)

Cost is per token, both input and output — this makes cost roughly proportional to context length and volume of calls.

Gain

- ▶ Access to frontier-quality models
- ▶ No hardware to buy or maintain
- ▶ Fast to prototype with
- ▶ Provider handles scaling

Give up

- ▶ Your data leaves your control (Section 6)
- ▶ Model can be silently updated or deprecated (Lecture 2, Slide 44)
- ▶ Ongoing per-call cost, not fixed
- ▶ Rate limits can bottleneck large studies

The reproducibility problem is structural, not accidental: “we used GPT-4” is not a fixed method statement the way “we fit a logistic regression” is (Lecture 2, Slide 44). The p_θ you queried may not exist in that exact form when someone tries to replicate you.

- ▶ **Data governance:** sensitive data (health records, personal identifiers) never leaves infrastructure you control — often a hard requirement, not a preference (Section 6).
- ▶ **Reproducibility:** once you have the weights, they don't change under you. You can archive the exact model alongside your data and code.
- ▶ **Cost at scale:** for very high call volumes, a fixed hardware cost can beat an unbounded per-token bill.
- ▶ **Full control:** you can inspect, modify, or fine-tune the model, and audit its behaviour directly.

Trade-off, stated plainly: open-weight models you can realistically self-host generally lag frontier closed models on many tasks — though the gap has been narrowing quickly.

What local hosting actually requires

- ▶ **Hardware:** the binding constraint is usually GPU memory (VRAM). Larger models need more of it to hold their parameters.
- ▶ **Quantization:** a technique to shrink a model's memory footprint by storing weights at lower numerical precision (e.g. 16-bit $\rightarrow$ 4-bit).
 - ▶ Trade-off: smaller footprint and faster inference, at some cost in output quality — usually modest, but task-dependent.
 - ▶ This is what makes running a large open-weight model on a single workstation GPU feasible at all.
- ▶ **Serving tools:** software that loads the weights and exposes them as something you can query — e.g. Ollama (simplest, good for prototyping), vLLM or llama.cpp (more control, better throughput).

Realistic framing: this is now accessible to a competent non-specialist, but still requires more setup than an API key.

	Hosted API	Local hosting
Best model quality	Usually yes	Usually lags
Sensitive data	Requires strong agreements	Full control
Reproducibility	Weak (model can change)	Strong (fixed weights)
Cost shape	Per-call, scales with use	Fixed hardware cost
Setup effort	Minimal	Moderate–high

Bridge forward: the “sensitive data” row is not a minor entry in this table — for much of the work in this room (health records, demographic microdata), it can be the decision that overrides every other consideration. That’s Section 6.

Every time you send data to an LLM — via an API call, or even a locally hosted model on shared infrastructure — you are making a data-handling decision with the same weight as choosing where to store a dataset.

- ▶ This is not hypothetical for this room: health records, demographic microdata, parliamentary/political text with identifiable individuals, survey responses with free-text fields.
- ▶ Ethics approvals, data sharing agreements, and funder requirements (UKRI, NHS, ONS) were almost certainly written *before* LLM APIs existed as a research tool — so the obligations don't disappear, they just don't mention this explicitly.
- ▶ **Default stance:** the burden is on you to show a given use is compliant, not on the tool to warn you it isn't.

When you send a prompt to a hosted API, ask three concrete questions:

1. **Is it logged?** Most providers retain request data for some period, for abuse monitoring or debugging, even if not used for training.
2. **Is it used for training?** Consumer-tier products often reserve the right to use conversations to improve models, unless you opt out. Enterprise/API tiers typically do *not* train on your data by default — but “typically” is not a guarantee; read the actual terms.
3. **Where does it live?** Data may be processed or stored outside the UK/EU, which has direct implications for GDPR and institutional data residency requirements.

Zero Data Retention (ZDR) agreements exist with some providers: contractually, your data is not stored or logged at all beyond the immediate request. These typically require an enterprise agreement, not a personal account.

Before sending anything to a model, classify it. Categories that should raise a flag:

- ▶ **Health data:** diagnoses, clinical notes, prescriptions — special category data under GDPR
- ▶ **Direct identifiers:** names, addresses, NHS/NI numbers, precise dates of birth
- ▶ **Indirect identifiers:** combinations of variables (postcode + age + occupation) that could re-identify someone even without a name
- ▶ **Data on minors** or other populations with heightened protection requirements
- ▶ **Anything your ethics approval explicitly restricts** in terms of storage location or third-party processing

Rule of thumb: if you would need to redact it before putting it in a public GitHub repo, be equally cautious about putting it in an API call.

1. **Can the data be adequately anonymised or synthetically substituted** before it touches any model, hosted or local? If yes, do this first — it resolves most of the problem regardless of which model you use.
2. **If not, does your institution have an enterprise agreement with strong contractual protections** (no training use, no retention, appropriate data residency) with a given API provider? If yes, this may be sufficient — confirm with your data protection officer, not just the provider's marketing page.
3. **If no such agreement exists, or the data is highly sensitive regardless**, local hosting (Section 5) on institutionally-approved infrastructure is likely required.
4. **When genuinely uncertain**, treat it as case (3) until your ethics board or data governance office confirms otherwise — the conservative default costs you time; the alternative can cost a great deal more.

Local hosting isn't always necessary if you're careful about what actually reaches the model:

- ▶ **Pre-process before sending:** strip or pseudonymise identifiers programmatically before constructing the prompt, and only re-attach them locally afterward.
- ▶ **Aggregate where possible:** if your task doesn't need individual-level text, send summaries or aggregated features instead.
- ▶ **Use enterprise/API tiers, not consumer chat interfaces,** for anything beyond casual exploration — consumer products' data policies are typically less protective and less contractually binding.
- ▶ **Document your reasoning** (Section 7) — if a reviewer or ethics board later asks why a given approach was compliant, you want a written answer already prepared, not reconstructed after the fact.

Recall the reproducibility issue from Lecture 2 (Slide 44): “we used GPT-4” is not a fixed method statement the way “we fit a logistic regression” is.

- ▶ Classical methods sections assume the tool is stable and fully specified by name (a t-test is a t-test).
- ▶ LLMs are not stable in this sense (Slide 44), and — as Section 3 showed — often not even a *single* model, but an orchestrated system.
- ▶ Journals are catching up: PNAS, Nature-family journals, and others now require explicit AI-use disclosure statements.

The good news: the ML community has already built reporting conventions for exactly this gap. We can borrow, rather than invent, a documentation habit.

Proposed as a short, standard document accompanying any released model, answering:
what is this model, and where should it and shouldn't it be used?

Core sections worth borrowing into a methods write-up:

- ▶ **Model details:** exact name, version, provider, date/architecture if known
- ▶ **Intended use:** what the model was designed and evaluated for
- ▶ **Factors and evaluation data:** what populations/conditions was it tested on, and how
- ▶ **Quantitative performance,** broken down by relevant subgroup where possible — not just an aggregate number
- ▶ **Ethical considerations and known limitations**

For your paper: you're not writing a full model card, but you should be able to answer these questions about whatever model you used.

Same idea, applied to **data** rather than models — directly relevant if you fine-tune, evaluate against a benchmark, or construct any dataset the LLM touches.

Key questions a datasheet asks:

- ▶ **Motivation:** why was this dataset created, and by whom?
- ▶ **Composition:** what does an instance represent? Are there sensitive attributes present?
- ▶ **Collection process:** how was the data collected, and over what timeframe?
- ▶ **Preprocessing/labelling:** what transformations were applied before you saw it?
- ▶ **Uses:** what has this dataset been used for, and what should it *not* be used for?

Relevance here: if you evaluate an LLM against a benchmark (recall Lecture 2's contamination concern, Slide 27), you need to know this dataset's provenance to interpret the result at all.

Written specifically for language data, with an explicit goal of surfacing whose language a system was built and evaluated on — directly relevant to demographic and cross-cultural research (Lecture 2, Slide 45).

Recommended fields include:

- ▶ **Curation rationale:** why this text, and not other text?
- ▶ **Speaker/annotator demographics:** age, gender, language variety, socioeconomic background where known
- ▶ **Language variety:** which dialect(s), register(s), and languages are represented?
- ▶ **Speech situation:** written vs. spoken, planned vs. spontaneous, time period

Why this matters for you specifically: an LLM's pretraining corpus is essentially never accompanied by a data statement — so this is one of the sharpest instances of the validity gap from Lecture 2 (Slide 32): you often cannot know whose language shaped the model's behaviour on your population.

Synthesising the three conventions into something practical for a methods section or supplementary material:

1. **Model:** exact name, version/snapshot date, provider, access mode (API vs. local)
2. **Parameters:** temperature, max tokens, any system prompt (verbatim, in an appendix)
3. **Prompts:** the exact prompt template used, ideally verbatim
4. **Calls:** number of API calls, dates of calls, any retries/failures and how they were handled
5. **Validation:** how outputs were checked against ground truth or human judgment, and what metric was used (Lecture 2, Slide 34)
6. **Data handling:** confirmation of what data was sent, under what governance agreement (Section 6)
7. **Known limitations:** language/population mismatch, contamination risk, or anything from Section 4's benchmark caveats

Every section today traces back to the same structural fact from Bommasani et al.: a foundation model is general-purpose infrastructure, adapted — by you, or by whoever built the interface in front of you — into something specific.

- ▶ **Landscape (Sec. 2–3):** that infrastructure is usually a *system* of models, not one model, and its composition changes constantly.
- ▶ **Choice (Sec. 4):** which model(s) you adapt is a methodological decision with downstream consequences, not a default.
- ▶ **Access (Sec. 5) and governance (Sec. 6):** how you access that infrastructure has real costs, real risks, and real obligations attached.
- ▶ **Documentation (Sec. 7):** because none of this is fixed the way a t-test is fixed, writing it down is not bureaucratic overhead — it is what makes your result mean something to someone reading it in two years.